

Vulnerability Assessment Report

Target:

testphp.vulnweb.com

Read-Only Security
Assessment

Prepared By:

Samiyan Mansuri
Cyber Security Intern

Testing Approach: Read-Only / Passive Assessment

Assessment Overview

Executive Summary

Scope & Methodology

Tools Used

Risk Analysis

Key Findings

Recommendations

Conclusion

Testing Approach: Read-Only / Passive Assessment

Assessment Objective

Assessment Objective

The objective of this vulnerability assessment was to identify potential security weaknesses in the target web application using a strictly read-only and passive testing approach.

The assessment focused on detecting misconfigurations, missing security controls, and exposure risks that could increase the attack surface of the application.

No exploitation was performed during this assessment.

Testing Approach: Read-Only / Passive Assessment

Testing Approach

This assessment followed ethical security testing practices and avoided any intrusive techniques. Only publicly accessible components of the website were analyzed.

The testing methodology included passive scanning, security header analysis, and configuration review to evaluate the overall security posture of the application.

Assessment Scope & Tools

Assessment Scope

This vulnerability assessment focused exclusively on publicly accessible components of the target website. The engagement followed a strictly read-only approach to ensure no disruption to the live environment.

The assessment included security header evaluation, service exposure review, and misconfiguration detection to identify potential risks.

Out of Scope: Exploitation, brute force attacks, authentication bypass, and denial-of-service testing were not performed.

Tools Used

Nmap — Port & service discovery

OWASP ZAP — Passive vulnerability scanning

Browser DevTools — Security header analysis

Testing Approach: Read-Only / Passive Assessment

Security Testing Methodology

Our Contact

✓ Testing Process

- Target Identification
- Port & Service Enumeration (Nmap)
- Passive Vulnerability Scanning (OWASP ZAP)
- Security Header Analysis
- Misconfiguration Detection
- Risk Classification

🔍 Methodology Overview

The vulnerability assessment was conducted using a structured and industry-aligned methodology focused on identifying security weaknesses without performing intrusive testing.

A passive approach was followed to analyze publicly accessible components and evaluate the overall security posture of the application.

Testing Approach: Read-Only / Passive Assessment

Risk Classification Framework

Risk Levels Defined

High Risk

Vulnerabilities that could significantly impact confidentiality, integrity, or availability of the system and require immediate remediation.

Medium Risk

Weaknesses that may be exploited under certain conditions and should be addressed in a timely manner.

Low Risk

Issues that pose minimal direct risk but should be resolved to improve overall security posture.

Vulnerability Summary

Assessment Findings Summary

The vulnerability assessment identified multiple security weaknesses primarily related to missing security headers and configuration gaps.

No critical vulnerabilities were discovered; however, several medium-risk issues require timely remediation to reduce the potential attack surface.

Severity	Count	Priority
Medium	4	High
Low-Medium	2	Moderate
Low	Several	recommended

Medium-risk vulnerabilities should be prioritized to enhance the overall security posture of the application.

Absence of Anti-CSRF Tokens

● **Medium**

Description

The assessment identified that certain application forms do not implement Anti-CSRF tokens. This security control is essential to prevent unauthorized commands from being transmitted from a user that the web application trusts.

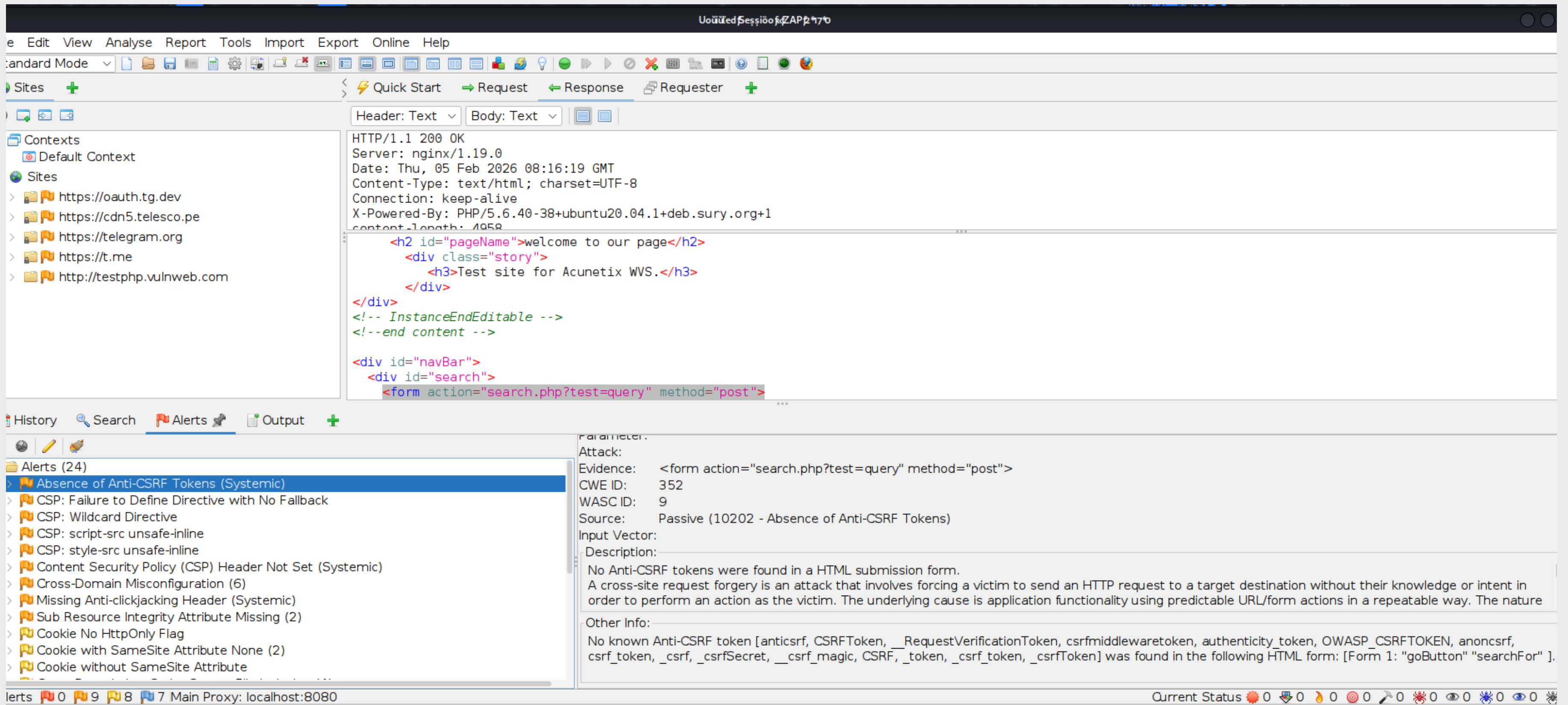
Impact

An attacker could potentially trick authenticated users into submitting malicious requests, leading to unauthorized actions within the application.

Recommendation

Implement unique Anti-CSRF tokens for all sensitive requests and validate them on the server side to ensure request authenticity.

Proof Of Concept (POC)



Your paragraph text

Missing Content Security Policy (CSP) Header

● Medium

Description

The assessment revealed that the web application does not enforce a Content Security Policy (CSP). CSP is a critical security mechanism that helps prevent attacks such as Cross-Site Scripting (XSS) by restricting which resources can be executed by the browser.

Impact

Without a properly configured CSP header, attackers may be able to inject malicious scripts into web pages, potentially leading to data theft, session hijacking, or website defacement.

Recommendation

Implement a strong Content-Security-Policy header that allows resources only from trusted domains and disables unsafe inline scripts.

Proof Of Concept (POC)

[illegible]

Your paragraph text

Missing Anti-Clickjacking Header

● Medium

Description

The assessment identified that the application does not include anti-clickjacking protections such as the X-Frame-Options or Content-Security-Policy frame-ancestors header. These headers prevent the website from being embedded within malicious iframes.

Impact

Attackers could trick users into clicking hidden elements by loading the website inside a malicious frame. This may lead to unintended actions being performed without the user's knowledge.

Recommendation

Configure one of the following security headers:

X-Frame-Options: DENY

Content-Security-Policy: frame-ancestors 'none';

Proof Of Concept (POC)

Uoüüed Sessão de ZAP 170

e Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Sites +

Quick Start Request Response Requester +

Header: Text Body: Text

Contexts

- Default Context

Sites

- https://oauth.tg.dev
- https://cdn5.telesco.pe
- https://telegram.org
- https://t.me
- http://testphp.vulnweb.com

HTTP/1.1 200 OK
Server: nginx/1.19.0
Date: Thu, 05 Feb 2026 08:16:19 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1
content-length: 4958

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html><!-- InstanceBegin template="/Templates/main_dynamic_template.dwt.php" codeOutsideHTMIsLocked="false" -->
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-2">

<!-- InstanceBeginEditable name="document_title_rgn" -->
<title>Home of Acunetix Art</title>
<!-- InstanceEndEditable -->
<link rel="stylesheet" href="style.css" type="text/css">
<!-- InstanceBeginEditable name="headers_rgn" -->

History Search Alerts Output +

Alerts (24)

- Absence of Anti-CSRF Tokens (Systemic)
- CSP: Failure to Define Directive with No Fallback
- CSP: Wildcard Directive
- CSP: script-src unsafe-inline
- CSP: style-src unsafe-inline
- Content Security Policy (CSP) Header Not Set (Systemic)
- Cross-Domain Misconfiguration (6)
- Missing Anti-clickjacking Header (Systemic)
- Sub Resource Integrity Attribute Missing (2)
- Cookie No HttpOnly Flag
- Cookie with SameSite Attribute None (2)
- Cookie without SameSite Attribute

Source: Passive (10020 - Anti-clickjacking Header)
Alert Reference: 10020-1
Input Vector:
Description:
The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

Other Info:

Solution:
Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.
If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never

Alerts 0 9 8 7 Main Proxy: localhost:8080

Current Status 0 0 0 0 0 0 0 0 0 0 0 0

Your paragraph text

Cross-Origin Resource Sharing (CORS) Misconfiguration

● Medium

Description

The assessment identified a misconfigured Cross-Origin Resource Sharing (CORS) policy on the web server. CORS is a security mechanism that controls how resources on a web server are shared with external domains.

Impact

If improperly configured, unauthorized third-party websites may be able to access sensitive data from the application. This could increase the risk of data exposure.

Recommendation

Restrict the Access-Control-Allow-Origin header to trusted domains only and avoid using overly permissive configurations such as wildcard (*) policies.

Proof Of Concept (POC)

Standard Mode

Sites

Contexts

Sites

Quick Start

Request

Response

Requester

Header: Text

Body: Text

Content-Length: 20489

Last-Modified: Mon, 03 Apr 2023 11:46:12 GMT

Connection: keep-alive

ETag: "642abc84-4ff5"

Expires: Mon, 09 Feb 2026 08:20:54 GMT

Cache-Control: max-age=345600

Access-Control-Allow-Origin: *

(function(window) {

(function(window){

window.__parseFunction = function(__func, __attrs) {

__attrs = __attrs || [];

__func = '(function(' + __attrs.join(',') + '){' + __func + '})';

return window.execScript ? window.execScript(__func) : eval(__func);

}

})(window));

(function(window){

function addEvent(el, event, handler) {

History

Search

Alerts

Output

CSP: Wildcard Directive

CSP: script-src unsafe-inline

CSP: style-src unsafe-inline

Content Security Policy (CSP) Header Not Set (Systemic)

Cross-Domain Misconfiguration (6)

Missing Anti-clickjacking Header (Systemic)

Sub Resource Integrity Attribute Missing (2)

Cookie No HttpOnly Flag

Cookie with SameSite Attribute None (2)

Cookie without SameSite Attribute

Cross-Domain JavaScript Source File Inclusion (4)

Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s) (Systemic)

Server Leaks Version Information via "Server" HTTP Response Header Field (Systemic)

Source: Passive (10098 - Cross-Domain Misconfiguration)

Input Vector:

Description:

Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server.

Other Info:

domain. Web browser implementations do not permit arbitrary third parties to read the response from authenticated APIs, however. This reduces the risk somewhat. This misconfiguration could be used by an attacker to access data that is available in an unauthenticated manner, but which uses some other form of security, such as IP address white-listing.

Solution:

Ensure that sensitive data is not available in an unauthenticated manner (using IP address white-listing, for instance).

Configure the "Access-Control-Allow-Origin" HTTP header to a more restrictive set of domains, or remove all CORS headers entirely, to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive manner.

Alerts

0

9

8

7

Main Proxy: localhost:8080

Current Status

0

0

0

0

0

0

0

0

0

0

0

Your paragraph text

Missing Subresource Integrity (SRI)

● Medium

Description

The assessment identified a misconfigured Cross-Origin Resource Sharing (CORS) policy on the web server. CORS is a security mechanism that controls how resources on a web server are shared with external domains.

Impact

If improperly configured, unauthorized third-party websites may be able to access sensitive data from the application. This could increase the risk of data exposure.

Recommendation

Restrict the Access-Control-Allow-Origin header to trusted domains only and avoid using overly permissive configurations such as wildcard (*) policies.

Proof Of Concept (POC)

The screenshot displays the Burp Suite application window. At the top, the menu bar includes File, Edit, View, Analyse, Report, Tools, Import, Export, Online, and Help. Below the menu is a toolbar with various icons for file operations and analysis. The main interface is divided into several panels:

- Sites Panel:** Located on the left, it shows a list of configured sites: https://oauth.tg.dev, https://cdn5.telesco.pe, https://telegram.org, https://t.me, and http://testphp.vulnweb.com.
- HTTP History Panel:** The central panel displays a list of HTTP requests. The selected request is from https://telegram.org, showing a 200 OK status. The response body contains HTML code, including a script tag for a Telegram widget.
- Alerts Panel:** Located at the bottom left, it lists various security alerts. The alert 'Sub Resource Integrity Attribute Missing (2)' is highlighted in blue.
- Details Panel:** On the bottom right, it provides detailed information about the selected alert, including the WASC ID (15), source (Passive), input vector, description, and solution.

The status bar at the bottom indicates the current status of the application, showing various icons and the text 'Current Status'.

Your paragraph text

Additional Security Observations (Low Risk)

Overview

During the assessment, several low-risk security gaps were identified. While these issues do not pose an immediate threat, addressing them will improve the overall defense posture of the application and align it with security best practices.

Risk Level: Low

Observations

- Server Version Exposure

The web server discloses its version information, which may help attackers identify known vulnerabilities.

- Missing Security Headers

Certain recommended headers were not observed, reducing protection against common web-based attacks.

- Cookie Configuration Weaknesses

Cookies may lack secure attributes such as HttpOnly or Secure flags.

- Information Disclosure via Headers

Excessive header information may reveal details about the underlying technology stack.

- Charset Not Explicitly Defined

Failure to specify a charset may create inconsistencies in how browsers interpret content.

Recommendation

It is recommended to follow secure configuration best practices, disable unnecessary information disclosure, and implement all standard security headers to strengthen the application's resilience against potential threats.

Security Recommendations

Overview

Based on the findings from the vulnerability assessment, several security improvements are recommended to reduce risk exposure and strengthen the overall security posture of the application.

Recommended Actions

✓ Implement Security Headers

Configure headers such as Content-Security-Policy, X-Frame-Options, and HttpOnly cookies.

✓ Enable Anti-CSRF Protection

Use unique tokens to validate legitimate user requests.

✓ Restrict Cross-Origin Access

Allow resource sharing only with trusted domains.

✓ Adopt Secure Configuration Practices

Disable unnecessary server information and enforce secure cookie attributes.

✓ Enforce HTTPS

Encrypt all communications using SSL/TLS to protect sensitive data.

✓ Perform Regular Security Testing

Conduct periodic vulnerability assessments to proactively identify risks.

Proactive remediation significantly reduces the likelihood of successful cyber attacks.

Assessment Conclusion

Conclusion

The vulnerability assessment identified several security gaps primarily related to missing protective headers and configuration weaknesses. While no critical vulnerabilities were discovered, the presence of multiple medium-risk issues indicates an increased attack surface that should be addressed promptly.

Implementing the recommended security measures will significantly enhance the application's resilience against modern web-based threats and align it with industry security best practices.

Security is an ongoing process, and proactive monitoring is essential to maintaining a strong security posture.

Prepared by Samiyan Mansuri
Cyber Security Intern